

Added ObjectDataProvider gadget generation for MessagePack (Typeless)

Added ObjectDataProvider gadget generation for MessagePack (Typeless) - Dane Evans (nines-nine), ysoserial.net project.

- Published: 2023-03-13
- Original: <<https://github.com/pwntester/ysoserial.net/pull/146>>
- Preserved from: <https://github.com/pwntester/ysoserial.net/pull/146> (github-api) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Added ObjectDataProvider gadget generation for MessagePack (Typeless)

- Repository: pwntester/ysoserial.net
- Opened by: nines-nine
- Opened: 2023-03-13
- State: closed

Body

MessagePack when in 'Typeless' mode (using the TypelessContractlessStandardResolver) will serialize type information along with the data. It uses property setters to initialize the objects it's deserializing. This allows use of the ObjectDataProvider gadget to achieve code execution.

The approach I've used is somewhat odd, but is due to some limitations of being able to control the properties that are serialized. There doesn't appear to be a mechanism to exclude properties (or even private fields) from serialization when using Typeless mode, so my approach is to create a surrogate object graph of the ObjectDataProvider in a dynamic assembly, then use reflection to modify MessagePack's internal type cache so that the surrogate objects are serialized with the real AssemblyQualifiedNames of ObjectDataProvider, Process and ProcessStartInfo.

When deserializing this object graph it acts on the real ObjectDataProvider gadget and deserializes into a code execution scenario.

Comments

nines-nine, 2023-03-13

Also, I've had to update the ysoserial project to use .NET 4.7.2 as MessagePack isn't supported on v4.5.2. I'm not sure if this causes any other issues in the wider project by breaking other gadgets etc. The lack of unit tests means that this might require some input from somebody with a better understanding of the implications of this upgrade.

irsdl, 2023-03-13

I don't think there is anything against using .NET 4.7.2 but historically we have tried to keep the .NET Framework version as old as possible so it will be compatible with more applications. I haven't looked at the changes yet but is there perhaps a blog post explaining the technique you have used and explained here or do you plan to publish something about it? There is always a good story behind gadgets so I am interested to see what made you to come up with this whether it was a vulnerable target or just pure curiosity.

@pwntester , shall we accept the changes as long as they work and perhaps apply any further improvements/structure changes later if required?

nines-nine, 2023-03-13

I don't think there is anything against using .NET 4.7.2 but historically we have tried to keep the .NET Framework version as old as possible so it will be compatible with more applications.

I figured this and from my knowledge of the current set of gadgets I assumed that it wouldn't break anything we currently have.

I haven't looked at the changes yet but is there perhaps a blog post explaining the technique you have used and explained here or do you plan to publish something about it?

I'm intending to publish a blog post on the technique I had to use and I can reference it in the additional reading section once I've put it together. There were several options for how to generate a working ObjectDataProvider gadget but most of them had some form of drawback.

There is always a good story behind gadgets so I am interested to see what made you to come up with this whether it was a vulnerable target or just pure curiosity.

I've seen a few examples in the wild of using MessagePack's 'Typeless' mode. My guess is that it's utilized as a more performant, smaller output version of BinaryFormatter. I've also seen several examples of threads where the OP has read the MSDN guidance on BinaryFormatter and people have suggested MessagePack as an alternative.

I do have a potentially interesting MessagePack RCE in the works which doesn't use any of the gadgets available in ysoserial.net. I actually had to write a tool to process the assemblies of the application and look for a suitable gadget. I've not had a chance to work on a PoC for it yet and so it's still undisclosed, but it'll use the same approach I've had to use here.

nines-nine, 2023-03-13

Just a point on improvements. I'd be interested to see if there's a better approach than the one I've taken. Obviously I'll hash out exactly why I took this approach in the blog post, but realistically I ran through quite a few options.

1. Updating MessagePack to support type name substitutions

(Not really appropriate for normal users)

1. Forking MessagePack with the above functionality

(Maintenance hell)

1. Using an older version of MessagePack which had similar functionality

(Potential breaking changes, might not support new formats)

1. Patching MessagePack in-memory to intercept the type name building method with my own implementation

(Absolute insanity. I actually started implementing this.)

1. Manually overwriting portions of the binary output

(Slow, prone to errors, several offsets and byte counts need to be modified, even worse when the payload is compressed, obviously wouldn't be integrated to ysoserial.net)

1. Programmatically overwriting portions of the binary output

(Doesn't work when the payload is compressed, MessagePack will optimize out some type names)

1. Hard-coding the surrogate class and serializing that instead

(MessagePack will optimize out type information for adjacent classes in the same assembly. Would require a whole DLL dedicated to surrogate classes for this one generator.)

You get the idea.

pwntester, 2023-03-14

@pwntester , shall we accept the changes as long as they work and perhaps apply any further improvements/structure changes later if required?

That sounds good to me. I took a quick look and the code looks ok too. @irsdl feel free to merge it after reviewing.

irsdl, 2023-03-14

The gadget works fine for me so all is ok. For improvement, it might be better to set the default encoding to base64 for them as the output contains byte data. This can be simply done by changing the `base64Default` variable in

<https://github.com/pwntester/ysoserial.net/blob/b2366494c442d06b2be336f2d015af2baa6c7bc9/ysoserial/Program.cs#L564> to:

```
List<String> base64Default = new List<string>() { "BinaryFormatter", "ObjectStateFormatter", "MessagePackTypeless",
```

Also for the credit part, @NinesPsygnosis's name should be added to the `Contributors` of `ObjectDataProviderGenerator` at least. I suggest to also add a note on top of the helper module (`ysoserial/Helpers/MessagePackObjectDataProviderHelper.cs`) too.

These changes can be applied later so I will just merge the PR for now.

irsdl, 2023-03-14

I forgot to mention that the README also needs updating now.

nines-nine, 2023-03-14

Also for the credit part, @NinesPsygnosis's name should be added to the `Contributors` of `ObjectDataProviderGenerator` at least. I suggest to also add a note on top of the helper module (`ysoserial/Helpers/MessagePackObjectDataProviderHelper.cs`) too.

I'm actually already listed there (Dane Evans) as I also wrote the initial implementation for SharpSerializer gadget generation.

chudyPB, 2023-03-16

Hi!

Good job with this commit @NinesPsygnosis! TBH, I wanted to contribute with the MessagePack too, but it seems that I'm late to the party. It's a good thing though, because your code is much nicer than mine.

I wanted to add one important thing about the MessagePack. `ObjectDataProvider` gadget allows you to call arbitrary methods (and leads to RCE) from `MessagePack 2.3.75` . In earlier versions, it in fact leads to DoS. But let me explain from the beginning.

MessagePack tends to strongly change its behaviour. Before version `2.3.75` , it by default called protected setters: even if the member was not given in the serialized payload. Of course, it was setting the member to `null` .

So, `ObjectDataProvider` extends the `DataSourceProvider` . It has a following member:

```

protected Dispatcher Dispatcher
{
    get
    {
        return this._dispatcher;
    }
    set
    {
        if (this._dispatcher != value)
        {
            this._dispatcher = value;
        }
    }
}

```

According to that, older versions of `MessagePack` will set the `Dispatcher` member to null.

When we use the `ObjectDataProvider` to call the method on an instantiated object (only feasible approach for the `MessagePack`, at least I think so), it refreshes the object two times. The method (here, `Process.Start`) is executed on the second refresh. However, every refresh is finished with the `OnQueryFinished` method.

```

protected virtual void OnQueryFinished(object newData, Exception error, DispatcherOperationCallback completi
{
    Invariant.Assert(this.Dispatcher != null); // [1]
    if (this.Dispatcher.CheckAccess())
    {
        this.UpdateWithNewResult(error, newData, completionWork, callbackArguments);
        return;
    }
    ...

```

At [1], the `Invariant.Assert` is called. In our case, `false` value will be provided as an input, because `Dispatcher` is equal to null.

```
internal static void Assert(bool condition)
{
    if (!condition)
    {
        Invariant.FailFast(null, null);
    }
}
```

According to that, the `ObjectDataProvider` gadget in older versions of `MessagePack` will call the `Invariant.FailFast`, which will basically kill the current process.

To sum up, I guess that you should add a note that this gadgets work for `MessagePack > 2.3.75` (or potentially from version `2.2.113`).

Now the very good information - `ObjectDataProvider` with the `System.Diagnostics.Process` should not lead to DoS! Older versions of `MessagePack` should not be able to deserialize the `Process`, thus it should throw an exception during the deserialization and it should not call the `ObjectDataProvider` setters. According to that, your payload should be universal - leads to RCE on newer versions, does nothing on older ones.

If you want to test the DoS on older versions - just changed the `ObjectInstance` to some simple type that can be easily deserialized and try to call any method. It should do the work.

I'm not sure, but maybe you may want to implement a DoS gadget for the `MessagePack`? I guess that it could be generated with a simple switch, like `--dos (ysoserial.exe -f MessagePack -g ObjectDataProvider --dos)`. If you would like me to help you with that - not a problem.

Cheers and keep up the good work!

nines-nine, 2023-03-16

Hi @chudyPB thanks for the writeup. I've just tested the code I wrote and I can see that versions of `MessagePack` prior to `2.3.75` will not generate working payloads for the reason you described. I tried `2.2.113` but to no avail. It's cool to see you were also looking at this serializer. I should have a blog post up pretty soon which details my approach for generating the gadget this way, but for those familiar with this work the code should be self explanatory.

I'm not sure whether `ysoserial.net` is intended to generate non-RCE payloads or not. There are some interesting gadgets I've found in the past which don't give you the ability to run arbitrary code, but allow for other interesting behaviours. Given the conventions of the tool (i.e. specifying commands, etc) I'm not sure they'd be included and I figure the same probably goes for denial-of-service payloads.

I have an open PR that drastically reduces the necessary code to generate the ODP gadget, so I'll push another change to the README that references which versions this is applicable to.

nines-nine, 2023-03-23

@chudyPB I'd like to include the details about the limitations in older versions of MessagePack in my blog post. Are you happy for me to include these? I'll add attribution if you like.

chudyPB, 2023-03-23

Sure, you can do that. :)

Archived from <https://github.com/pwntester/ysoserial.net/pull/146>. Rendered offline from the local Markdown copy.